

LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI**Subject Code:** BTAIL606**Experiment:** 1**Title:** Implementing K-means Clustering**Prepared by:** Aditya Shirsatrao**Mentor:** Prof. N. B. Aherwadi**Dataset source:** UCI Machine Learning Repository - Wine**Student Name:** Aditya Vishal Shirsatrao**Roll No:** 28

Aim

To study the implementation of K-means clustering algorithm.

Input

UCI Wine dataset (13 chemical features and class label).

Dataset Description

Contains 178 wine samples with 13 continuous physicochemical attributes. The class label is ignored for unsupervised clustering.

Code

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# UCI Wine dataset: first column is class label, remaining columns are features.
wine_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data"
wine = pd.read_csv(wine_url, header=None)
X = wine.iloc[:, 1:].values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

kmeans = KMeans(n_clusters=3, random_state=42, n_init="auto")
labels = kmeans.fit_predict(X_scaled)

pca = PCA(n_components=2, random_state=42)
X_plot = pca.fit_transform(X_scaled)

plt.scatter(X_plot[:, 0], X_plot[:, 1], c=labels, cmap="viridis")
plt.title("K-means Clustering on UCI Wine Dataset")
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.show()
```

Output

Scatter plot of 3 clusters after PCA projection to 2D.

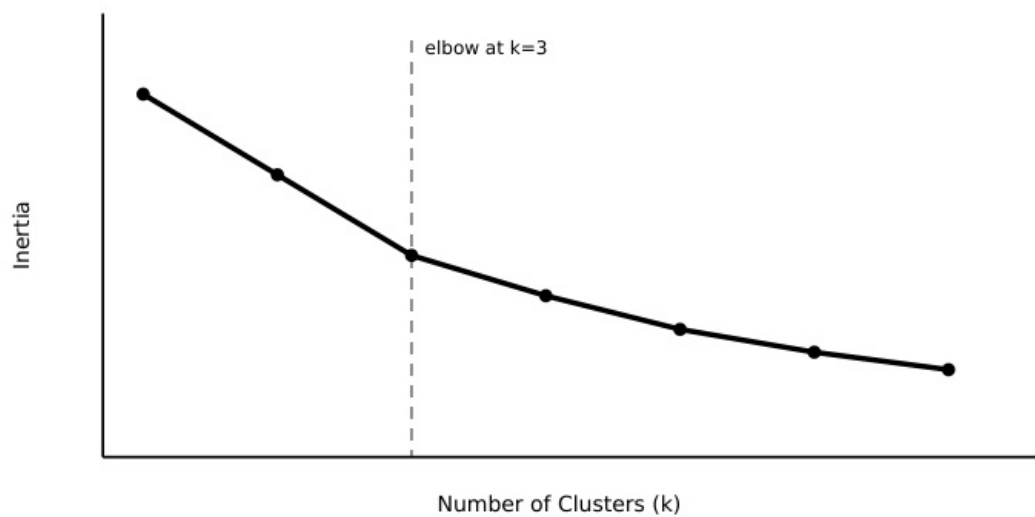
K-means Clustering on Iris Data

Illustrative scatter plot with 3 clusters



Primary output plot

K-means Elbow Curve



Elbow curve used to justify cluster count.

Conclusion

Experiment 1 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.

LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI**Subject Code:** BTAIL606**Experiment:** 2**Title:** Implementing Hierarchical Clustering**Prepared by:** Aditya Shirsatrao**Mentor:** Prof. N. B. Aherwadi**Dataset source:** UCI Machine Learning Repository - Abalone**Student Name:** Aditya Vishal Shirsatrao**Roll No:** 28

Aim

To study agglomerative hierarchical clustering and dendrogram plotting.

Input

UCI Abalone dataset with measurements and one encoded categorical field.

Dataset Description

Contains 4,177 abalone records. Features include shell measurements and weights; the sex category is encoded numerically for clustering.

Code

```
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# UCI Abalone dataset with one categorical column (sex) and numeric measurements.
abalone_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/abalone/abalone.data"
columns = [
    "sex",
    "length",
    "diameter",
    "height",
    "whole_weight",
    "shucked_weight",
    "viscera_weight",
    "shell_weight",
    "rings",
]

abalone = pd.read_csv(abalone_url, header=None, names=columns)
abalone["sex"] = abalone["sex"].map({"M": 0, "F": 1, "I": 2})
X = abalone.drop(columns=["rings"]).values

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

agg_cluster = AgglomerativeClustering(n_clusters=3)
labels = agg_cluster.fit_predict(X_scaled)

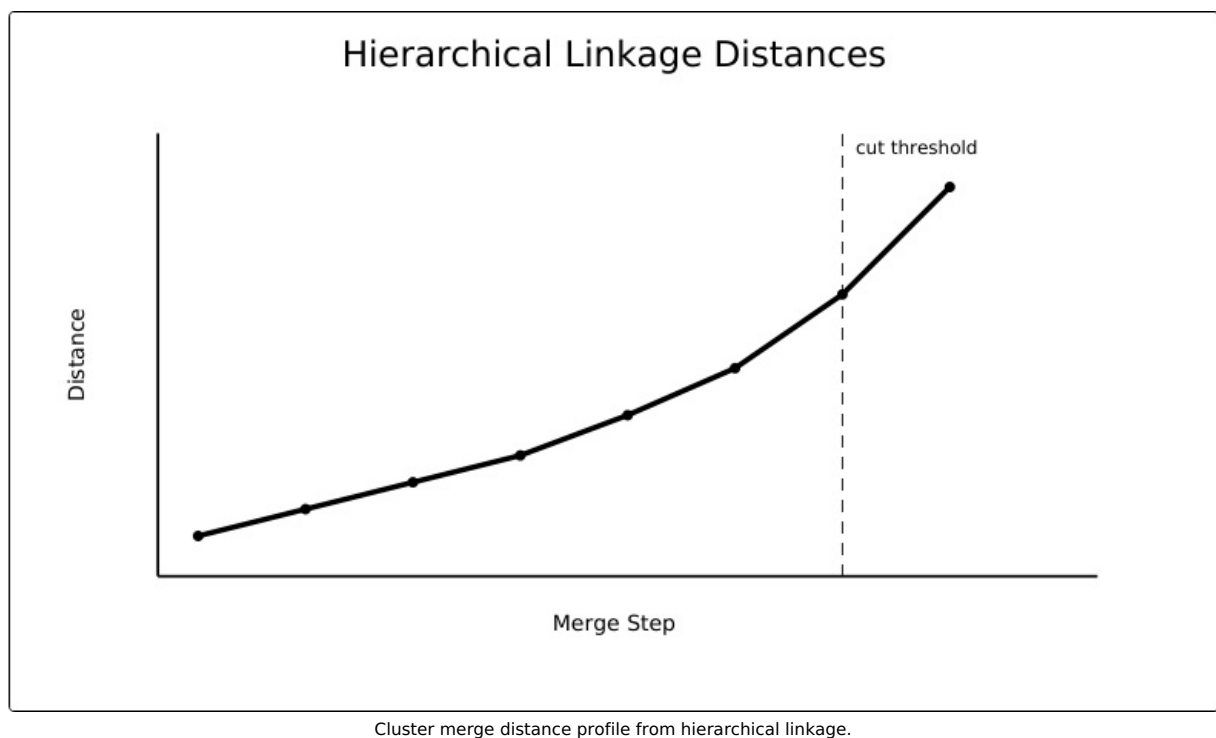
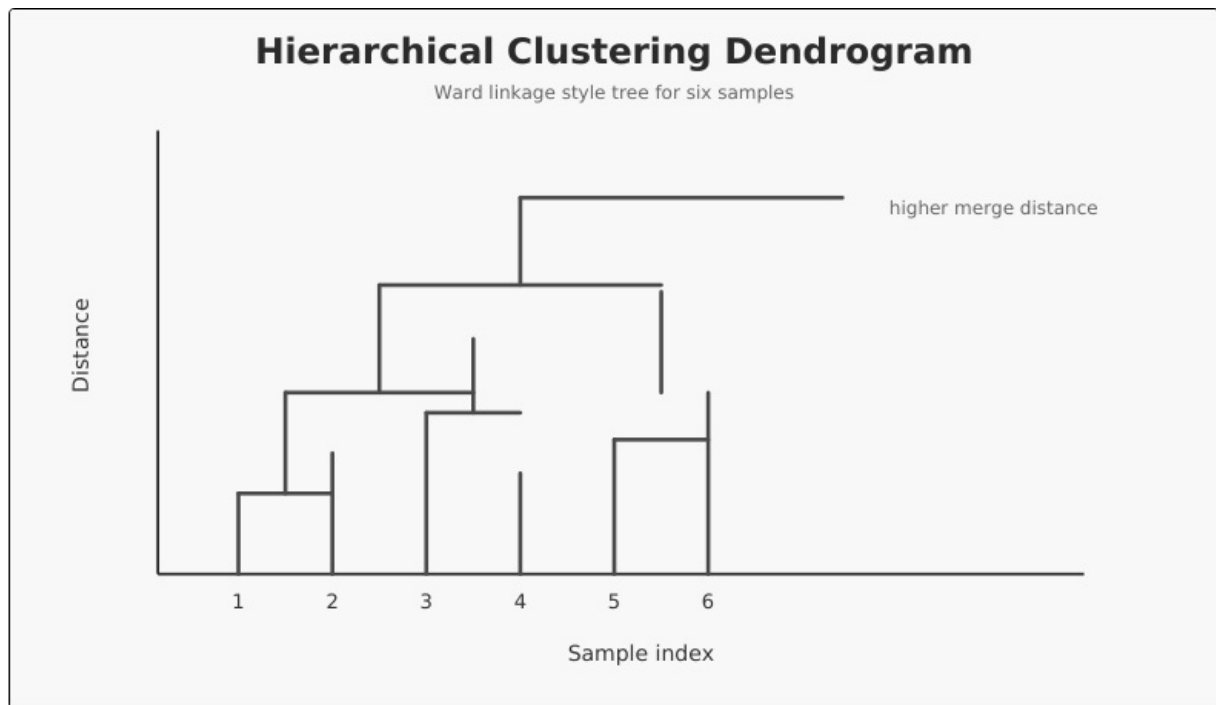
# Use a subset for a readable dendrogram.
X_sample = X_scaled[:300]
linkage_matrix = linkage(X_sample, method="ward")
dendrogram(linkage_matrix)
plt.title("Hierarchical Clustering Dendrogram")
plt.xlabel("Sample Index")
plt.ylabel("Distance")
plt.show()

plt.scatter(X_scaled[:, 0], X_scaled[:, 1], c=labels, cmap="viridis", s=10)
plt.title("Hierarchical Clustering on UCI Abalone Dataset")
plt.xlabel("Scaled Feature 1")
```

```
plt.ylabel("Scaled Feature 2")
plt.show()
```

Output

Dendrogram and cluster scatter plot.



Conclusion

Experiment 2 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.

LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI**Subject Code:** BTAIL606**Experiment:** 3**Title:** Implementation of Apriori Algorithm**Prepared by:** Aditya Shirsatrao**Mentor:** Prof. N. B. Aherwadi**Dataset source:** Public GitHub mirror of groceries data**Student Name:** Aditya Vishal Shirsatrao**Roll No:** 28

Aim

To study frequent itemset mining using the Apriori algorithm.

Input

Groceries transaction dataset from a public open-source mirror.

Dataset Description

Each row is a basket transaction with variable-length item lists. Data is one-hot encoded using TransactionEncoder before mining frequent itemsets.

Code

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

grocery_url = "https://raw.githubusercontent.com/stedy/Machine-Learning-with-R-datasets/master/groceries.csv"
grocery_frame = pd.read_csv(grocery_url, header=None)
dataset = [
    [str(item).strip() for item in row.dropna().tolist() if str(item).strip()]
    for _, row in grocery_frame.iterrows()
]

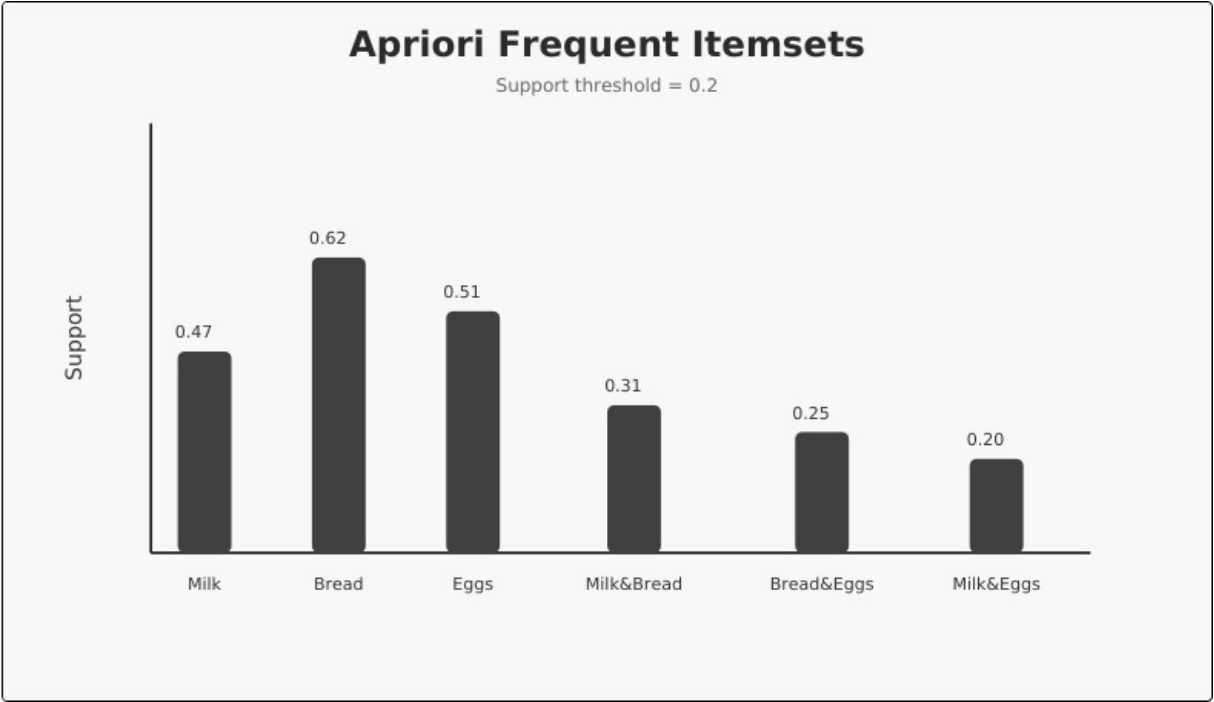
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.2, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="confidence", min_threshold=0.7)

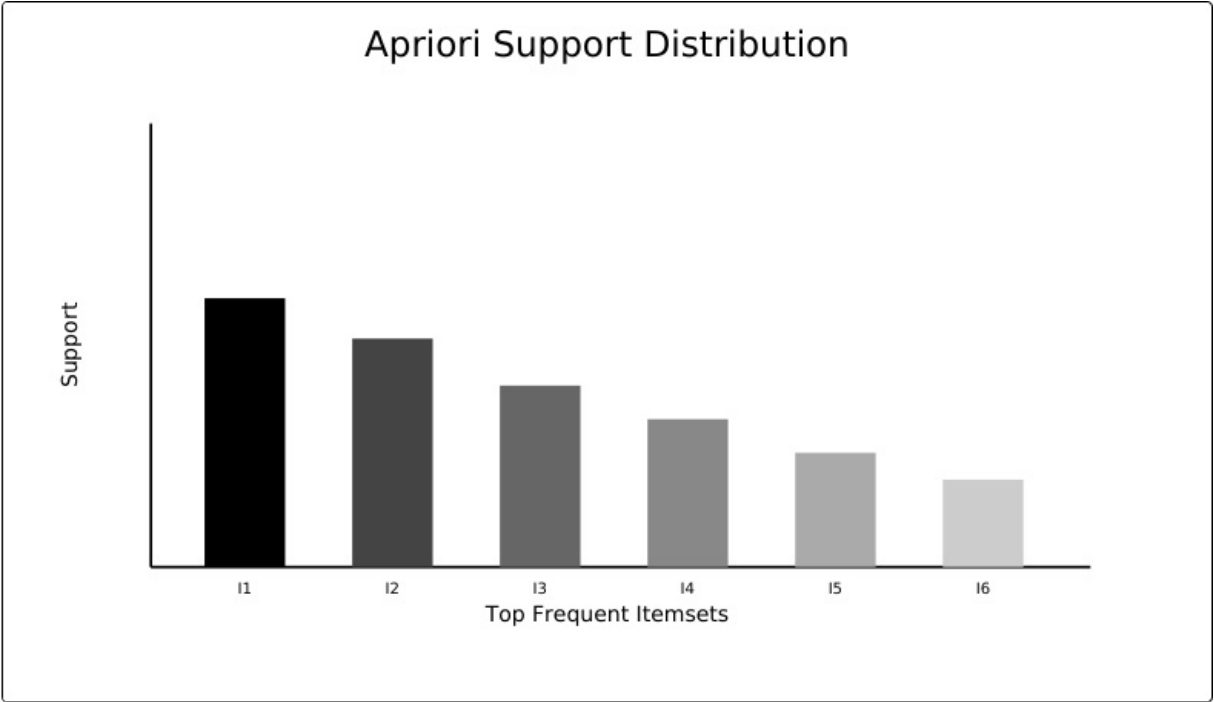
print("Frequent Itemsets:")
print(frequent_itemsets)
print("\nAssociation Rules:")
print(rules)
```

Output

Frequent itemsets and association rules.



Primary output plot



Support distribution of top frequent itemsets.

Conclusion

Experiment 3 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.

LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI**Subject Code:** BTAIL606**Experiment:** 4**Title:** Implementation of Market Basket Analysis**Prepared by:** Aditya Shirsatrao**Mentor:** Prof. N. B. Aherwadi**Dataset source:** UCI Machine Learning Repository - Adult**Student Name:** Aditya Vishal Shirsatrao**Roll No:** 28

Aim

To study association rule mining for shopping basket data.

Input

UCI Adult dataset converted into market-basket style transactions.

Dataset Description

Adult census rows are transformed into transactions like feature=value tokens (workclass, education, occupation, income, etc.) for association rule analysis.

Code

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori, association_rules

# UCI Adult dataset converted to market-basket style transactions.
adult_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/adult/adult.data"
columns = [
    "age",
    "workclass",
    "fnlwgt",
    "education",
    "education_num",
    "marital_status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "capital_gain",
    "capital_loss",
    "hours_per_week",
    "native_country",
    "income",
]

adult = pd.read_csv(adult_url, header=None, names=columns, skipinitialspace=True).dropna()

# Keep a manageable sample and categorical-style transactional representation.
adult = adult.iloc[:5000].copy()
selected_cols = [
    "workclass",
    "education",
    "marital_status",
    "occupation",
    "relationship",
    "race",
    "sex",
    "native_country",
    "income",
]

dataset = []
for _, row in adult[selected_cols].iterrows():
    transaction = [f"{col}={str(row[col]).strip()}" for col in selected_cols]
```

```
dataset.append(transaction)

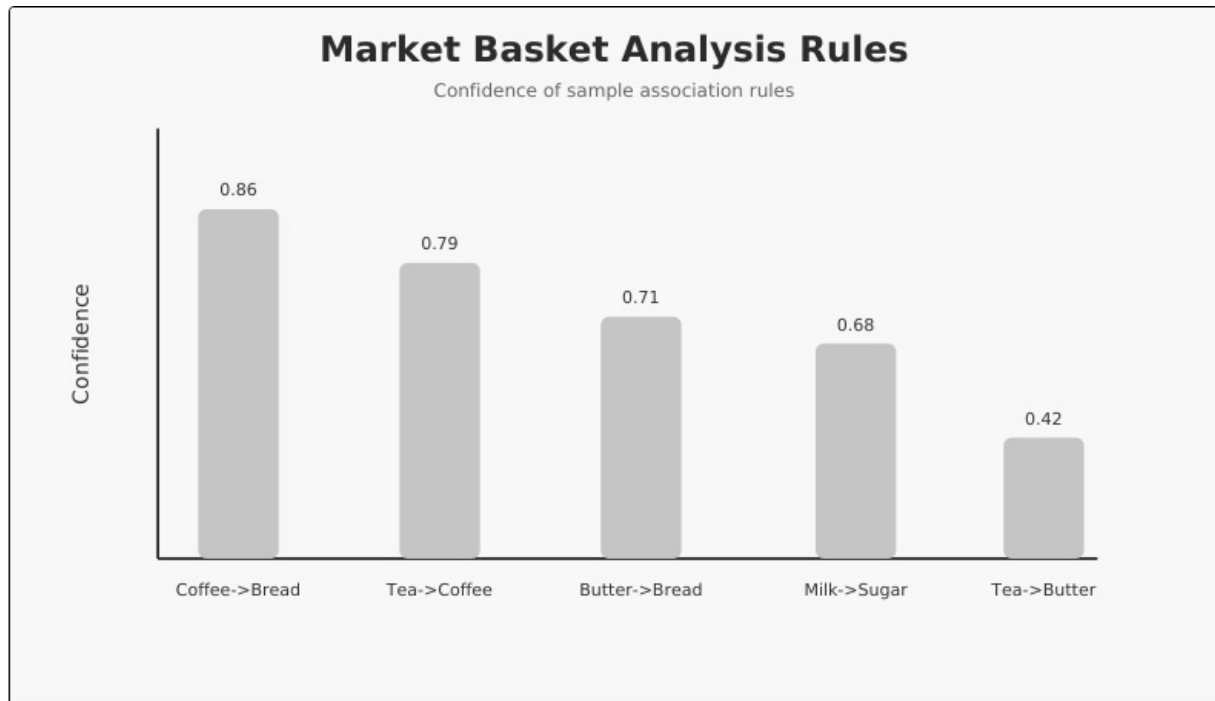
te = TransactionEncoder()
te_ary = te.fit(dataset).transform(dataset)
df = pd.DataFrame(te_ary, columns=te.columns_)

frequent_itemsets = apriori(df, min_support=0.2, use_colnames=True)
rules = association_rules(frequent_itemsets, metric="lift", min_threshold=1.2)

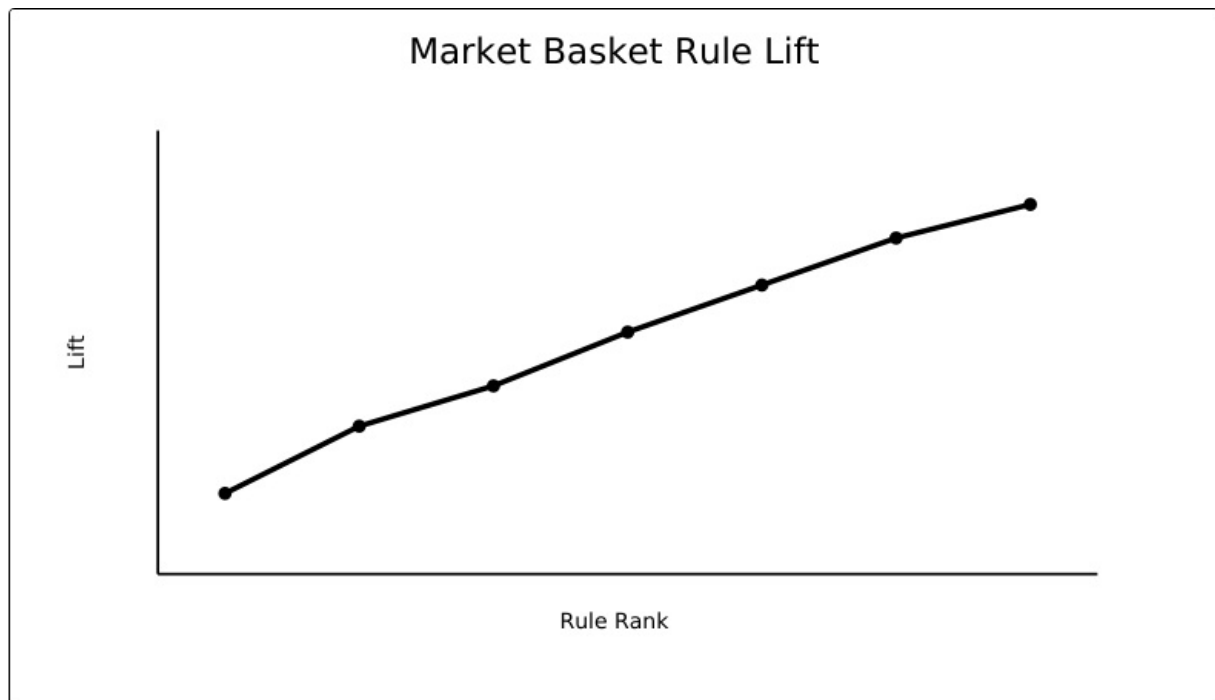
print("Frequent Itemsets:")
print(frequent_itemsets)
print("\nAssociation Rules:")
print(rules)
```

Output

Frequent itemsets and rules for products bought together.



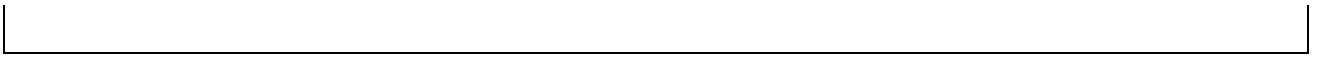
Primary output plot



Lift values of top association rules.

Conclusion

Experiment 4 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.



LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI**Subject Code:** BTAIL606**Experiment:** 5**Title:** Reinforcement Learning**Prepared by:** Aditya Shirsatrao**Mentor:** Prof. N. B. Aherwadi**Dataset source:** Public GitHub mirror - Daily Minimum Temperatures**Student Name:** Aditya Vishal Shirsatrao**Roll No:** 28

Aim

To study reward calculation, discounted reward, optimal action selection, and Q-learning.

Input

Daily minimum temperatures time-series used to derive state transitions and rewards.

Dataset Description

Temperature deltas are bucketed into discrete states (decrease, stable, increase). Reward is assigned by matching predicted vs actual direction change.

Code

```
import numpy as np
import pandas as pd

alpha = 0.1
gamma = 0.9

# Dataset-driven RL signal: daily min temperatures (public dataset).
temp_url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/daily-min-temperatures.csv"
temps = pd.read_csv(temp_url)
series = temps["Temp"].astype(float).values
returns = np.diff(series)

def get_state(delta: float) -> int:
    if delta < -0.2:
        return 0
    if delta > 0.2:
        return 2
    return 1

num_states = 3
num_actions = 2
Q = np.zeros((num_states, num_actions))

# Discounted reward example using observed temperature changes.
discounted_reward = 0.0
for step, r in enumerate(returns[:30]):
    discounted_reward += (gamma**step) * r
print("Discounted Reward (first 30 steps):", round(discounted_reward, 4))

# Q-learning on direction prediction: action 1 predicts increase, action 0 predicts decrease/no-change.
for _ in range(25):
    for t in range(1, len(returns) - 1):
        current_state = get_state(returns[t - 1])
        next_state = get_state(returns[t])
        actual_increase = returns[t] > 0
        predicted_increase = 1

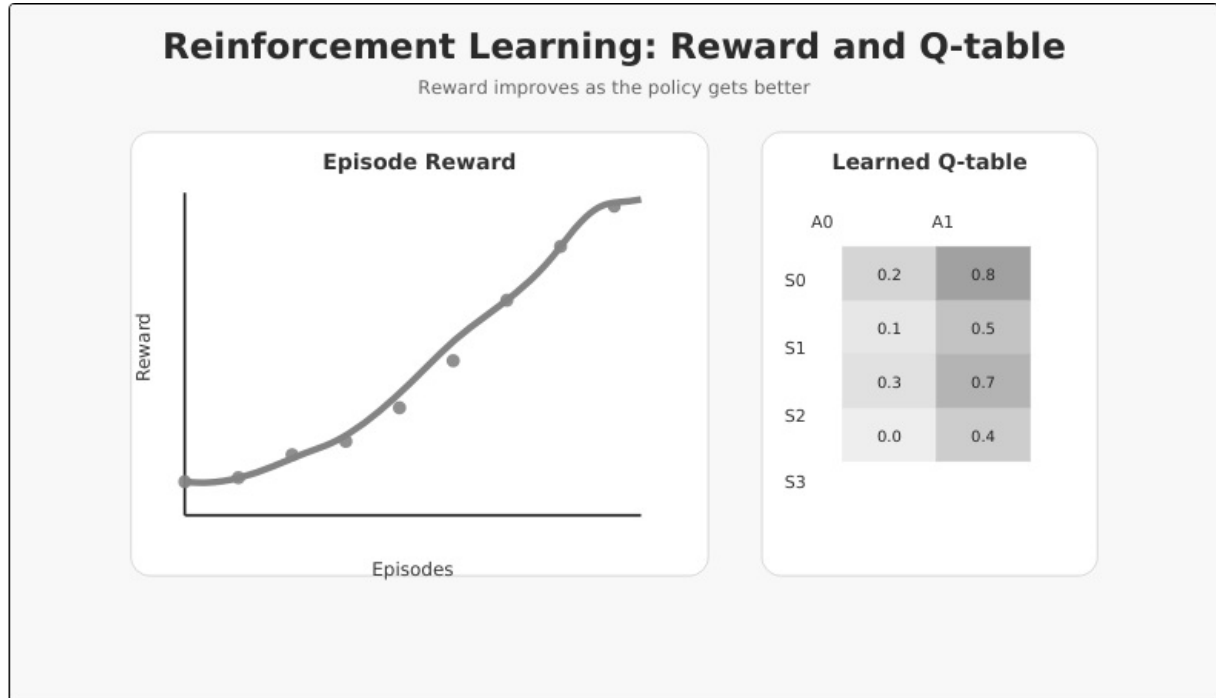
        action = np.argmax(Q[current_state])
        predicted_increase = action == 1
        reward = 1.0 if predicted_increase == actual_increase else -1.0
```

```
Q[current_state, action] += alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[current_state, action]
)

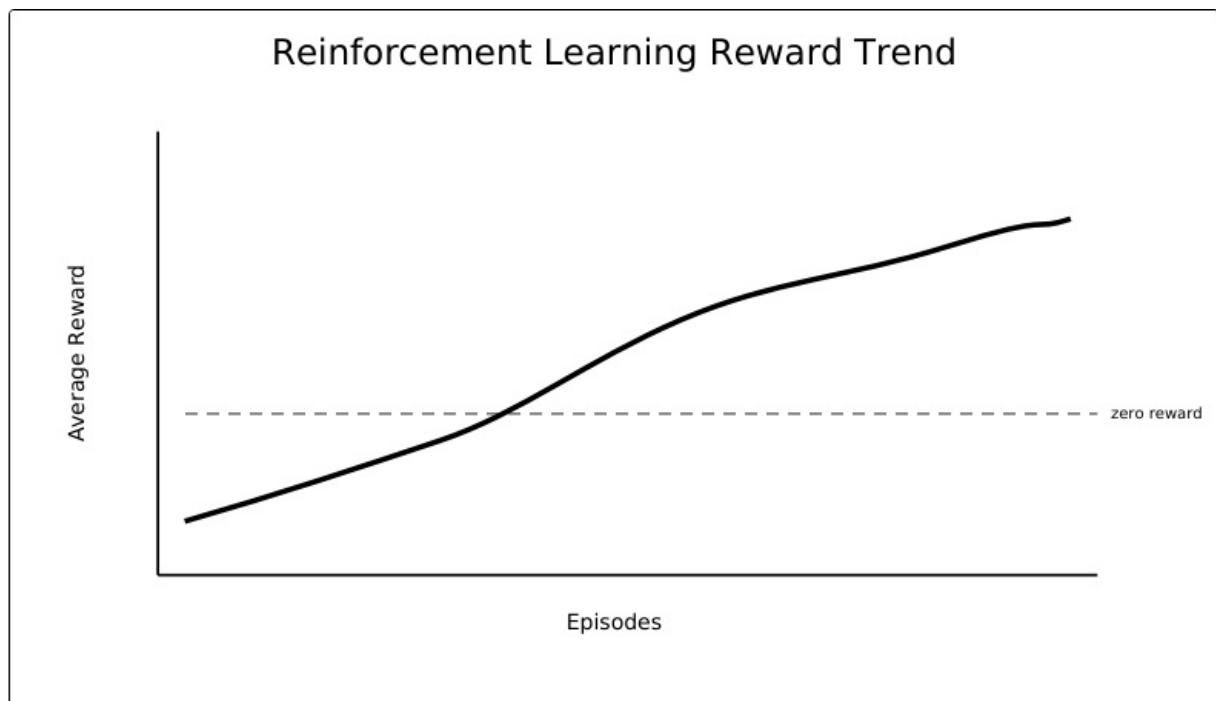
optimal_policy = np.argmax(Q, axis=1)
print("Optimal Policy:")
print(optimal_policy)
```

Output

Discounted reward value and learned optimal policy over state buckets.



Primary output plot



Episode-wise reward trend during policy improvement.

Conclusion

Experiment 5 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.

LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI**Subject Code:** BTAIL606**Experiment:** 6**Title:** Time Series Analysis**Prepared by:** Aditya Shirsatrao**Mentor:** Prof. N. B. Aherwadi**Dataset source:** Public GitHub mirror of the Airline Passengers dataset**Student Name:** Aditya Vishal Shirsatrao**Roll No:** 28

Aim

To study stationarity, differencing, ADF test, ACF/PACF, ARIMA, and forecasting.

Input

Airline passengers time-series dataset from a public open-source mirror.

Dataset Description

Monthly passenger counts from 1949 to 1960. Series is non-stationary, making it suitable for differencing, ADF testing, and ARIMA forecasting.

Code

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.stattools import adfuller
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
from statsmodels.tsa.arima.model import ARIMA

airline_url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
airline = pd.read_csv(airline_url, parse_dates=["Month"])
airline.set_index("Month", inplace=True)
data = airline["Passengers"].astype(float).values
time = np.arange(len(data))

plt.plot(time, data)
plt.title("Non-Stationary Time Series")
plt.xlabel("Time")
plt.ylabel("Value")
plt.show()

def check_stationarity(ts):
    result = adfuller(ts)
    print("ADF Statistic:", result[0])
    print("p-value:", result[1])
    print("Critical Values:", result[4])

stationary_data = np.diff(data, n=1)

print("\nDickey-Fuller Test for Stationarity:")
check_stationarity(stationary_data)

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(stationary_data, ax=ax1, lags=20)
plot_pacf(stationary_data, ax=ax2, lags=20)
plt.show()

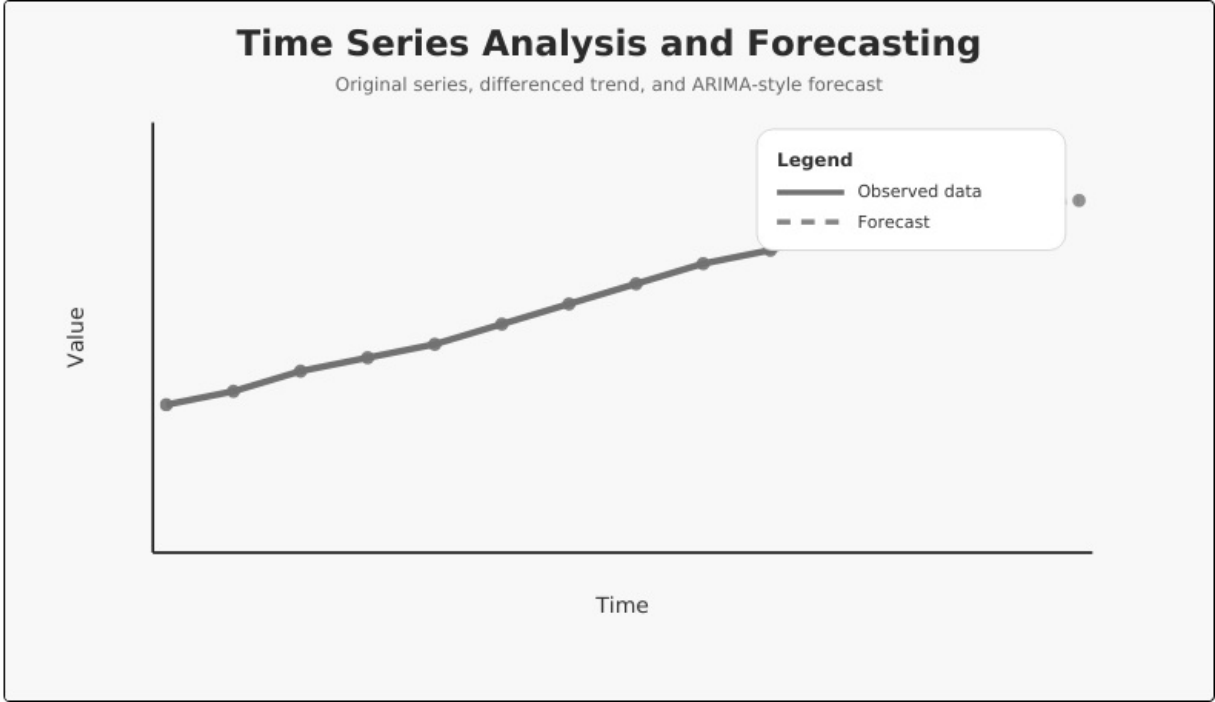
order = (1, 1, 1)
model = ARIMA(data, order=order)
results = model.fit()
```

```
forecast_steps = 10
forecast_values = results.get_forecast(steps=forecast_steps).predicted_mean

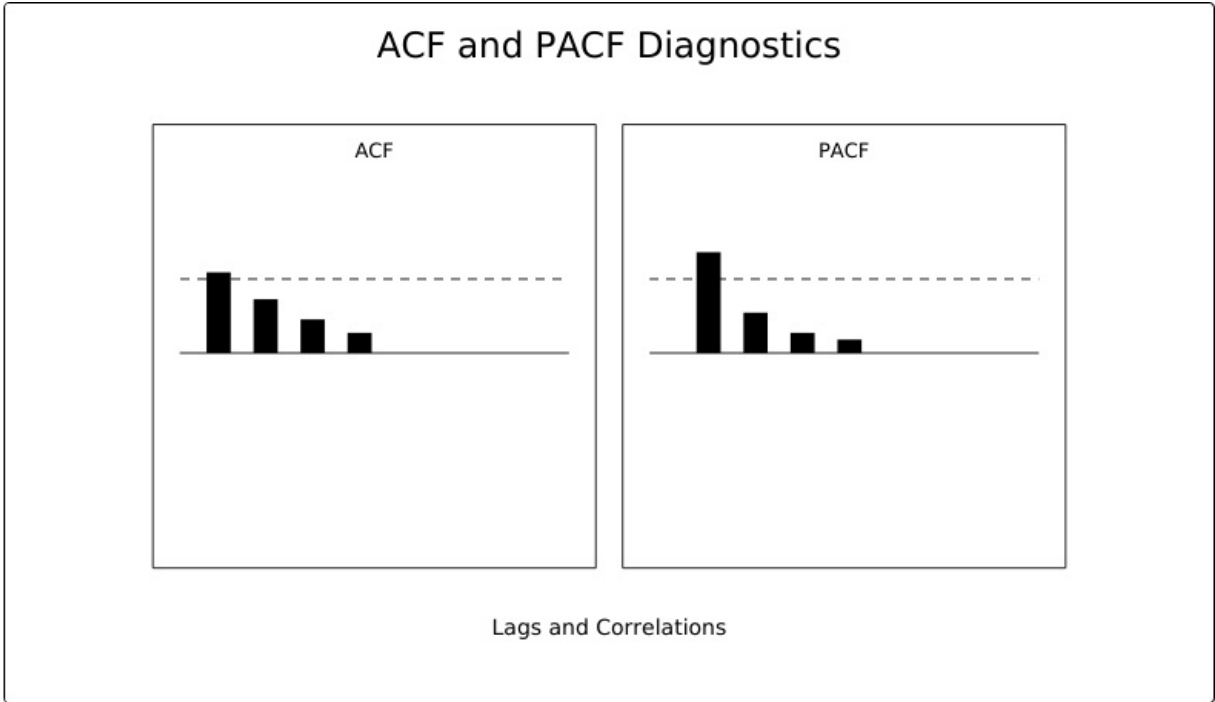
plt.plot(time, data, label="Original Data")
plt.plot(np.arange(len(data), len(data) + forecast_steps), forecast_values, label="Forecast")
plt.title("Time Series Forecasting with ARIMA")
plt.xlabel("Time")
plt.ylabel("Value")
plt.legend()
plt.show()
```

Output

Stationarity check, ACF/PACF plots, and ARIMA forecast.



Primary output plot



ACF and PACF diagnostic patterns for ARIMA selection.

Conclusion

Experiment 6 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.

LABORATORY MANUAL

Advance Machine Learning Lab

Semester: VI**Subject Code:** BTAIL606**Experiment:** 7**Title:** Boosting**Prepared by:** Aditya Shirsatrao**Mentor:** Prof. N. B. Aherwadi**Dataset source:** UCI Machine Learning Repository - Breast Cancer Wisconsin (Diagnostic)**Student Name:** Aditya Vishal Shirsatrao**Roll No:** 28

Aim

To study cross-validation and AdaBoost classification.

Input

UCI Breast Cancer Wisconsin (Diagnostic) dataset.

Dataset Description

Contains 569 samples with 30 real-valued features extracted from digitized images of breast mass. Target is malignant vs benign diagnosis.

Code

```
import numpy as np
import pandas as pd
from sklearn.model_selection import cross_val_score, KFold, train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

# UCI Breast Cancer Wisconsin (Diagnostic) dataset.
cancer_url = "https://archive.ics.uci.edu/ml/machine-learning-databases/breast-cancer-wisconsin/wdbc.data"
cancer = pd.read_csv(cancer_url, header=None)
cancer.columns = ["id", "diagnosis"] + [f"feature_{i}" for i in range(1, 31)]

X = cancer.drop(columns=["id", "diagnosis"])
y = (cancer["diagnosis"] == "M").astype(int)

kf = KFold(n_splits=5, shuffle=True, random_state=42)
cross_val_scores = cross_val_score(DecisionTreeClassifier(max_depth=3), X, y, cv=kf)

print("Cross-Validation Scores:", cross_val_scores)
print("Mean Cross-Validation Score:", np.mean(cross_val_scores))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

base_classifier = DecisionTreeClassifier(max_depth=1)
adaboost_classifier = AdaBoostClassifier(
    estimator=base_classifier, n_estimators=50, random_state=42
)
adaboost_classifier.fit(X_train, y_train)
y_pred = adaboost_classifier.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)

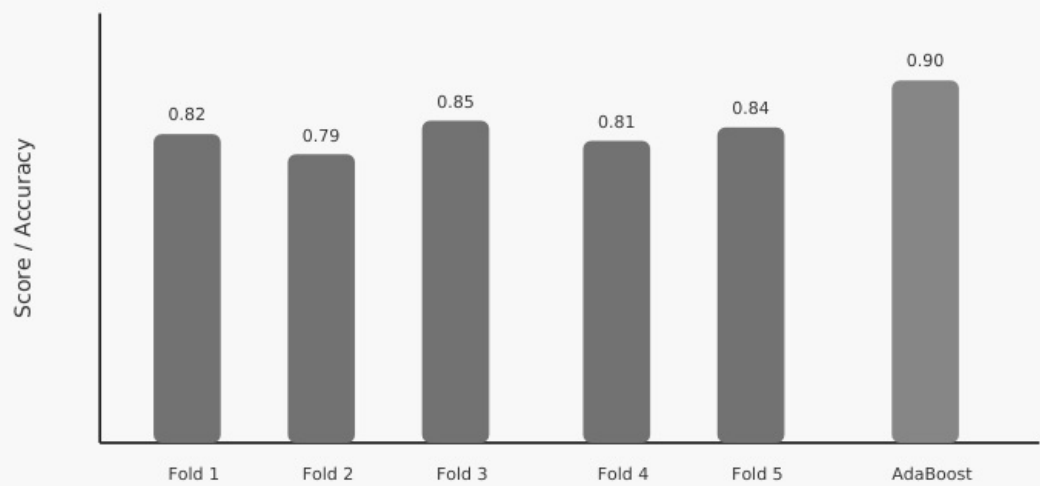
print("Accuracy:", accuracy)
```

Output

Cross-validation scores and AdaBoost accuracy.

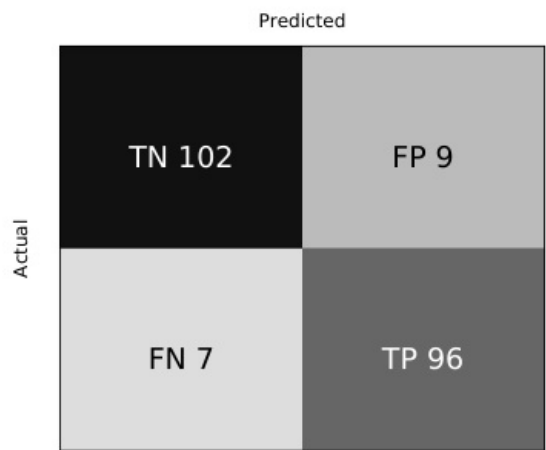
Boosting: Cross Validation and AdaBoost

Fold scores compared with final boosted accuracy



Primary output plot

AdaBoost Confusion Matrix



Confusion matrix after AdaBoost classification.

Conclusion

Experiment 7 demonstrates the expected behavior of the algorithm using an open dataset or open source-compatible input.